USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

Display Syscalls, and a Total System Freeze

UM-NATOS-016

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-016	1.0 · 2026-08-15	**Complete, verified on hardware**	Internal

1. Abstract

Two things happened after the display driver landed, and they belong in one report because the second was caused by the first and hidden by how the first was checked.

Applications can now draw. `SYS FILL`, `SYS TEXT` and `SYS DIMS` give bytecode access to the panel, confined to a per-application viewport by the same discipline that confines it to an arena. Containment is proved numerically in §5, not observed: **zero escapes across 136 fills** while a program did nothing but demand the entire screen.

Before any of that could be verified, the kernel had to be unfrozen. The display driver commit introduced a **total system halt** — not a slow display, a stopped clock — and it survived three commits and two "verified on hardware" claims because it was checked by looking at the panel rather than at the tick. §3 is that defect, and §3.4 is how it was missed, which is the more useful half.

2. Display syscalls

Three calls, added to the ISA of UM-NATOS-012 §3.

Syscall	Arguments	Effect
`FILL` (5)	`r0=x r1=y r2=w r3=h r4=colour`	Filled rectangle, viewport-relative
`TEXT` (6)	`r0=str r1=x r2=y r3=fg r4=bg r5=scale`	String from the arena
`DIMS` (7)	—	`r0 ← (width << 16) | height`

2.1 Viewports

Each application slot owns a horizontal strip: `y = 168 + id × 28`, 26 rows tall, full width. Above them the kernel keeps its status area; below them the colour strip. Neither is reachable from an application.

Coordinates are viewport-relative and the viewport's position is never disclosed. `DIMS` returns size only. A program is told how much canvas it has and nothing about where that canvas sits, which removes the temptation for a producer to compute absolute coordinates and removes the possibility of it naming one.

This is deliberately the arena model applied to a second resource. UM-NATOS-013 §5.2 established that an application cannot reach another's memory because an address outside its arena is *unrepresentable*. The same holds for pixels: a coordinate outside the viewport is clipped before it can exist.

2.2 Clipping belongs here, not in the driver

`display_fill_rect()` clips to the **panel**, which is the wrong boundary. A program allowed to paint anywhere on the panel could erase the kernel's status area, the colour strip, and every other application's output.

`vp_fill()` therefore clips to the **viewport**, in the offset domain, for exactly the reason `arena_contains()` does (UM-NATOS-010 §5.2). Coordinates arrive as `uint32_t`, so a program passing a negative value hands over something near `0xFFFFFFFF`; comparing it against the viewport width rejects it, whereas computing `x + w` first would wrap and admit it.

2.3 The first pointer to cross the boundary

`SYS TEXT` takes an arena offset, making it the first syscall to carry a pointer. It is handled the way `PUTS` already was: `copy_string()` walks the string one **bounds-checked byte at a time** into a 48-byte kernel buffer, and a string that runs to the end of the arena without a terminator faults rather than reading past it.

The copy is not redundant caution. `display_text()` takes a kernel pointer, and handing it an arena address would let the program's own stores change the string while it is being rendered.

Horizontal overrun is truncated to what fits; vertical overrun is refused outright, because half a line of text is worse than none and the glyph renderer has no partial-row mode.

2.4 A quantum bounds instructions, not milliseconds

`vm_run()` 's quantum bounds **instructions**. A display fill is one instruction and roughly 31 ms of bit-banged SPI, so a 2,000-instruction budget could be minutes of drawing inside a single call — and the preemption guarantee of UM-NATOS-012 §5 would be worthless.

Expensive syscalls now end the slice regardless of remaining budget, so wall-clock time per `vm_run()` stays bounded by roughly one display operation. Without it the host task disappears for minutes and every other application starves.

This is a general hazard, not a display one: any future syscall whose cost is measured in time rather than instructions needs the same treatment.

3. The freeze

3.1 Symptom

Boot completed. Every task entered. The shell printed its banner. The reporter ran the critical-section test and printed `PASS` at tick 8. Then nothing, ever again — no output, no scheduling, a frozen panel retaining its last frame.

```
before: 0 report lines in 35 s, ticks frozen at 8
after:  16 report lines, t=3246, all tasks healthy
```

3.2 Root cause

`task_yield()` set the comparator unconditionally:

```
xt_set_ccompare1(xt_ccount() + 64u);
```

The display task's frame delay was:

```
uint32_t until = timer_ticks() + 25u;
while (timer_ticks() < until) { task_yield(); }
```

`timer_ticks()` is a single load, so that loop runs in far fewer than 64 cycles. **Every pass pushed the comparator deadline further ahead of `CCOUNT`, so it was never reached and the timer interrupt stopped firing.**

That is not a slowdown. The tick drives every context switch in this kernel, so when it stops nothing else can run — and the task doing the yielding spins forever waiting for a clock it is itself preventing. A yield loop, the most innocuous-looking construct in the system, starved the mechanism that makes yielding mean anything.

3.3 Fix

The comparator is written only when the new deadline is **sooner** than the one already programmed:

```
uint32_t soon = xt_ccount() + 64u;
if ((int32_t)(soon - xt_get_ccompare1()) < 0) {
    xt_set_ccompare1(soon);
}
```

A yield can bring preemption forward. It can never defer it. The signed comparison handles `CCOUNT` wrapping.

3.4 How it survived three commits

This is the part worth keeping.

The display driver was verified by **looking at the panel** and asking whether it looked right, with the serial capture filtered down to boot-time lines. Sustained operation — the check applied to every previous milestone, and the one that produced every number in reports 008 through 015 — was not run. On that basis three commits were reported as sound, including one that said in as many words "regression: all self-tests still pass".

The status task draws a marker block that advances every frame, and UM-NATOS-015 §5 states its purpose explicitly: *"a display whose numbers all look plausible but never change is otherwise indistinguishable from a working one."* The instrument was built, documented, and then not read. It was frozen the entire time.

Worse, the question asked to confirm the driver — whether the colour bars were in the right order — is one a **frozen screen answers identically to a live one**. A static image was treated as evidence of a running system.

Bisection then briefly implicated the display syscall work, which was innocent: reverting to the previously "known-good" commit reproduced the freeze identically, and that is what showed the defect predated it.

A second claim in the same session — that the shell had stopped responding to commands — was also wrong, and was a fault in the test harness rather than the kernel. A probe either side of the console lock showed `execute()` reached and the lock taken immediately.

The common thread is not carelessness about any one fact. It is reporting on whatever evidence was nearest rather than on the evidence that would settle the question.

4. Design rule this establishes

> **A yield must never defer the clock it depends on.**

More generally: any routine that adjusts a deadline the scheduler depends on must only ever move it earlier. Deferring it, in a loop, disables preemption entirely — and does so silently, since the symptom appears wherever the system happened to stop rather than at the line responsible.

`_handler_level3`'s `PS.EXCM` rule (UM-NATOS-009 §6) is the other standing rule of this kind. Both share a shape: a single unconditional register write, correct in isolation, catastrophic under repetition.

5. Containment, measured

`gfxrogue` exists only to escape. It repeatedly asks to fill the entire 240×320 panel in red, then white, then from an origin of (60000, 60000) that arrives as a huge unsigned number.

```
vp calls=136  escapes=0  maxy=250/320
```

- **`escapes = 0`** — no rectangle reaching the driver ever fell outside the viewport it came from, across 136 fills.
- **`maxy = 250`** — the lowest row any application has painted. Slot 2's viewport is `168 + 2×28 = 224`, height 26, bottom edge **exactly 250**. Not one row below it, and nowhere near the panel's 320.

`vp_fill()` re-derives the absolute rectangle *after* clipping and re-checks it against the viewport before handing it to the driver, counting and refusing any that would fall outside. The duplication is deliberate: what matters is what actually reaches the panel, not what the clipping intended. If the two ever disagree, `escapes` stops being zero and says so.

5.1 Why this is a counter and not a photograph

Containment was visible on the panel, and the report back was "*I think it looks good.*"

That is a judgement, and this is the claim the entire syscall design rests on. Given §3.4, a hedged glance was not the evidence it deserved. The counter is also the more durable form: it keeps checking after everyone has stopped looking, and it distinguishes "confined" from "confined so far".

6. Metrics

Quantity	Value
Syscalls added	3
Application viewport	240×26, one per slot
Fills audited	136
Viewport escapes	**0**
Lowest row painted	250, of a 320-row panel
String buffer	48 B, per-byte bounds-checked
Native tasks	9
Image size	19,856 B
Commits the freeze survived	3

7. What this does not establish

- **No pixel-level drawing.** Rectangles and text only; no lines, circles, or bitmaps.
- **No image data from an application.** Nothing lets a program hand over a buffer of pixels, which is the next syscall to want and the next pointer to need bounds discipline.
- **No viewport negotiation.** Sizes are fixed by slot; an application cannot request more, and there is no way to grant a full-screen application anything.
- **No read-back.** A program cannot see what is on screen, including its own output.
- **No per-application draw accounting.** A program that draws constantly costs the display mutex and everyone's frame rate, and nothing measures or limits that.
- **The escape counter is not a proof of the clipping logic**, only evidence that it and the re-check agree on everything tried so far. A case both get wrong identically would pass.

8. References

- UM-NATOS-012 §3, §5 — the ISA and the quantum this extends
- UM-NATOS-013 §5.2 — the memory containment this mirrors for pixels
- UM-NATOS-010 §5.2 — offset-domain comparison, and why `x + w` is the wrong test
- UM-NATOS-015 §5 — the marker block, built to catch exactly the freeze in §3
- UM-NATOS-009 §6 — the other standing rule about unconditional register writes
- `kernel/task.c` — `task_yield()` and the earlier-only rule
- `kernel/vm.c` — `vp_fill`, `vp_text`, `copy_string`, and the escape counters